

Reguläre Ausdrücke

(Regular Expressions)

Theoretische Informatik

TH Lübeck

- NEAs und DEAs - gut zu implementieren, intuitiv zu verstehen und zu implementieren
- Abgeschlossen unter vielen Operationen

- Bis jetzt könne Sie die Anforderungen an die Passwörter durch Automaten beschreiben
- Problem: Als Eingabesprache für den Nutzer etwas lästig

- Bis jetzt könne Sie die Anforderungen an die Passwörter durch Automaten beschreiben
- Problem: Als Eingabesprache für den Nutzer etwas lästig
- Gute Nachricht: Das Problem ist schon lange bekannt und gelöst

- Bis jetzt könne Sie die Anforderungen an die Passwörter durch Automaten beschreiben
- Problem: Als Eingabesprache für den Nutzer etwas lästig
- Gute Nachricht: Das Problem ist schon lange bekannt und gelöst
- Lösung : reguläre Ausdrücke

- Bis jetzt könne Sie die Anforderungen an die Passwörter durch Automaten beschreiben
- Problem: Als Eingabesprache für den Nutzer etwas lästig
- Gute Nachricht: Das Problem ist schon lange bekannt und gelöst
- Lösung : reguläre Ausdrücke
 - Beschreiben ein Pattern für Strings (Wörter)
 - Bekannt von ...

- Bis jetzt könne Sie die Anforderungen an die Passwörter durch Automaten beschreiben
- Problem: Als Eingabesprache für den Nutzer etwas lästig
- Gute Nachricht: Das Problem ist schon lange bekannt und gelöst
- Lösung : reguläre Ausdrücke
 - Beschreiben ein Pattern für Strings (Wörter)
 - Bekannt von Unix grep oder Lexern wie Lex, Perl oder Java

Reguläre Ausdrücke

Beispiel (Deutsche Postleitzahlen)

```
if (Pattern.matches("\\d{5}", input)){  
    ...  
}
```

- `\d` steht für eine Ziffer.
- `{5}` bedeutet, dass der Ausdruck links davon 5 mal vorkommt.

- Geburtsdatum
- ...

- Induktionsanfang
 - $\emptyset$ ist ein regulärer Ausdruck
 - ε ist ein regulärer Ausdruck
 - a ist ein regulärer Ausdruck wenn a ein Buchstabe des Alphabets ist
- Induktionsschritt
 - Seien E und F reguläre Ausdrücke, dann auch
 - $(E + F)$
 - (EF)
 - E^*
- Konvention: Klammern wenn möglich einsparen, Konkatenation (EF) bindet stärker als Vereinigung $(E + F)$

Zu jedem regulären Ausdruck E ist eine Sprache $L(E)$ definiert durch

- Induktionsanfang
 - $L(\emptyset) = \emptyset$ (leere Sprache)
 - $L(\varepsilon) = \{\varepsilon\}$ Sprache mit leerem Wort
 - $L(a) = \{a\}$ Sprache mit Wort a
- Induktionsschritt
 - $L((E + F)) = L(E) \cup L(F)$
 - $L(EF) = L(E) \circ L(F)$
 - $L(E^*) = L(E)^*$

Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E,$

Beispiel (Natürliche Zahlen)

$$4 + 5 = 5 + 4$$

Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E,$

Beispiel (Sprachen)

$$a^*b + b^*a = b^*a + a^*b$$

$$a^*(b^* + c^*) = a^*(c^* + b^*)$$

Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E,$
- $E + (F + H) = (E + F) + H,$

Beispiel (Natürliche Zahlen)

$$4 + (5 + 6) = (4 + 5) + 6$$

Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E,$
- $E + (F + H) = (E + F) + H,$

Beispiel (Sprachen)

$$a^*b + (b^*a + c^*a) = (a^*b + b^*a) + c^*a$$

Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E,$
- $E + (F + H) = (E + F) + H,$
- $E(FH) = (EF)H,$

Beispiel (Sprachen)

$$a^*(b^*c^*) = (a^*b^*)c^*$$

Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E,$
- $E + (F + H) = (E + F) + H,$
- $E(FH) = (EF)H,$
- $E(F + H) = EF + EH$ and $(E + F)H = EH + FH.$

Beispiel (Natürliche Zahlen)

$$4(5 + 6) = 4 \cdot 5 + 4 \cdot 6$$

Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E,$
- $E + (F + H) = (E + F) + H,$
- $E(FH) = (EF)H,$
- $E(F + H) = EF + EH$ and $(E + F)H = EH + FH.$

Beispiel (Sprachen)

$$a^*(b^* + c^*) = a^*b^* + a^*c^*$$
$$(b^* + c^*)a^* = b^*a^* + c^*a^*$$

Identitäten

Seien E reguläre Ausdrücke. Dann gilt

- $\emptyset + E = E,$
- $\varepsilon E = E\varepsilon = E,$

Beispiel (Natürliche Zahlen)

$$4 + 0 = 4$$

$$1 \cdot 5 = 5$$

Identitäten

Seien E reguläre Ausdrücke. Dann gilt

- $\emptyset + E = E,$
- $\varepsilon E = E\varepsilon = E,$

Beispiel (Sprachen)

$$a^*b + \emptyset = a^*b$$

$$ba^*b\varepsilon = ba^*b$$

Identitäten

Seien E reguläre Ausdrücke. Dann gilt

- $\emptyset + E = E,$
- $\varepsilon E = E\varepsilon = E,$
- $\emptyset E = E\emptyset = \emptyset.$

Beispiel (Natürliche Zahlen)

$$4 \cdot 0 = 0$$

Identitäten

Seien E reguläre Ausdrücke. Dann gilt

- $\emptyset + E = E,$
- $\varepsilon E = E\varepsilon = E,$
- $\emptyset E = E\emptyset = \emptyset.$

Beispiel (Sprachen)

$$a^*b\emptyset = \emptyset$$

Regeln mit Stern

Sei E ein regulärer Ausdruck. Dann gilt

- $(E^*)^* = E^*$,

Beispiel (Sprachen)

$$(a^*)^* = a^*$$

Regeln mit Stern

Sei E ein regulärer Ausdruck. Dann gilt

- $(E^*)^* = E^*$,
- $\emptyset^* = \varepsilon$,

Regeln mit Stern

Sei E ein regulärer Ausdruck. Dann gilt

- $(E^*)^* = E^*$,
- $\emptyset^* = \varepsilon$,
- $\varepsilon^* = \varepsilon$,

Beispiel (Sprachen)

$$\varepsilon\varepsilon\varepsilon = \varepsilon$$

Regeln mit Stern

Sei E ein regulärer Ausdruck. Dann gilt

- $(E^*)^* = E^*$,
- $\emptyset^* = \varepsilon$,
- $\varepsilon^* = \varepsilon$,
- $E^+ \stackrel{\text{def}}{=} EE^*$,

Beispiel (Sprachen)

$$L((ab)^*) = \{\varepsilon, ab, abab, ababab, \dots\}$$

$$L((ab)^+) = \{ab, abab, ababab, \dots\}$$

Regeln mit Stern

Sei E ein regulärer Ausdruck. Dann gilt

- $(E^*)^* = E^*$,
- $\emptyset^* = \varepsilon$,
- $\varepsilon^* = \varepsilon$,
- $E^+ \stackrel{\text{def}}{=} EE^*$,
- $E^+ + \varepsilon = E^*$.

Beispiel (Sprachen)

$$L((ab)^*) = \{\varepsilon, ab, abab, ababab, \dots\}$$

$$L((ab)^+) = \{ab, abab, ababab, \dots\}$$

- Deutsche Postleitzahlen:

- Eigentlich:

- $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$

- $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$

- $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$

- kürzer mit ...-Notation: $(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)$

- Deutsche Postleitzahlen:
 - Eigentlich:
 $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$
 $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$
 $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$
 - kürzer mit ...-Notation: $(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)$
- Passwort muss Groß- und Kleinbuchstaben enthalten (Alphabet $\{a, A, b, B\}$ wegen der Einfachheit):

- Deutsche Postleitzahlen:

- Eigentlich:

- $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$

- $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$

- $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$

- kürzer mit ...-Notation: $(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)(0 + \dots + 9)$

- Passwort muss Groß- und Kleinbuchstaben enthalten (Alphabet $\{a, A, b, B\}$ wegen der Einfachheit):

- $(a + b)(a + b)^*(A + B)(a + b + A + B)^* + (A + B)(A + B)^*(a + b)(a + b + A + B)^*$

- Reguläre Ausdrücke werden schnell lang
- In der Praxis: “syntactic sugar” wie `\d` in Java
- Abkürzungen, um Ausdrücke kompakt aufschreiben zu können
- Damit kann man aber nicht “mehr” beschreiben
- Ausnahme: Backreferences
- Folgenden Konstruktionen sind kompakter, wenn man nur die grundlegenden regulären Ausdrücke betrachtet

- Reguläre Ausdrücke gut zum Beschreiben von Sprachen
- Wie implementiert man Prüfung ob Wort in Sprache des regulären Ausdrucks liegt?

- Reguläre Ausdrücke gut zum Beschreiben von Sprachen
- Wie implementiert man Prüfung ob Wort in Sprache des regulären Ausdrucks liegt?
- Die Lösung: Umwandlung in endlichen Automaten

Beispiel (Reguläre Ausdrücke in Java)

*The Pattern engine performs traditional **NFA-based** matching with ordered alternation as occurs in Perl 5.*

Äquivalenz von endlichen Automaten und regulären Ausdrücken

Satz (Kleene)

Die Menge der durch reguläre Ausdrücke beschreibbaren Sprachen ist genau die Menge der durch endliche Automaten beschreibbaren Sprachen.

Notation

Diese Sprachen heißen deshalb auch *reguläre Sprachen*.

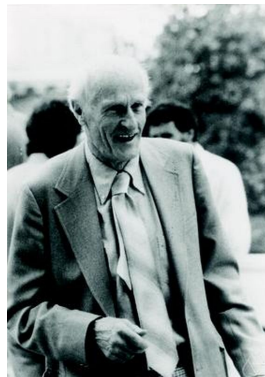


Abbildung 1: Stephen Cole Kleene

“Kleene, SC” by Konrad Jacobs, Lizenz: CC BY-SA 2.0 DE, Copyright: Mathematisches Forschungsinstitut Oberwolfach

Satz (Kleene - Hinrichtung)

Für jeden regulären Ausdruck E gibt es einen nichtdeterministischen endlichen Automaten A mit

$$L(E) = L(A).$$

Beweis.

Für einen regulären Ausdruck E kann ein NEA A mit $L(E) = L(A)$ induktiv konstruiert werden:

- Basis:



Beweis.

Für einen regulären Ausdruck E kann ein NEA A mit $L(E) = L(A)$ induktiv konstruiert werden:

- Basis:

- Automat für $\emptyset$:



Beweis.

Für einen regulären Ausdruck E kann ein NEA A mit $L(E) = L(A)$ induktiv konstruiert werden:

- Basis:

- Automat für $\emptyset$:



- Automat für ε :



Beweis.

Für einen regulären Ausdruck E kann ein NEA A mit $L(E) = L(A)$ induktiv konstruiert werden:

- Basis:

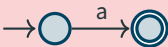
- Automat für $\emptyset$:



- Automat für ε :



- Automat für a :



Beweis.

Für einen regulären Ausdruck E kann ein NEA A mit $L(E) = L(A)$ induktiv konstruiert werden:

- Basis:

- Automat für $\emptyset$:



- Automat für ε :



- Automat für a :

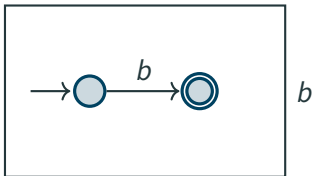
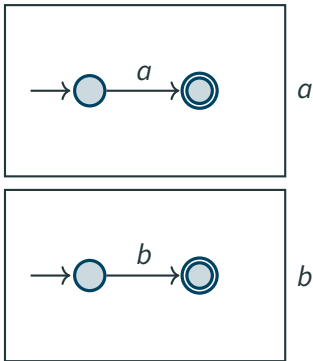


- Induktionsvoraussetzung: Für reguläre Ausdrücke E_1 und E_2 , gibt es NEAs A_1 und A_2 mit $L(A_1) = L(E_1)$ und $L(A_2) = L(E_2)$.

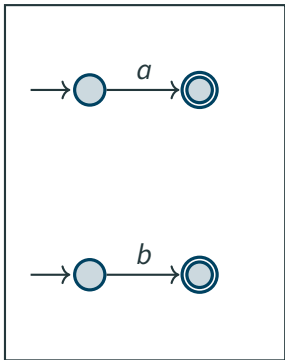
- Induktionsschritt:

- $E_1 + E_2$: Konstruiere NEA für Vereinigung von $L(A_1)$ und $L(A_2)$
- $E_1 \cdot E_2$: Konstruiere NFA für Konkatenation von $L(A_1)$ und $L(A_2)$
- E_1^* : Konstruiere NEA für $L(A_1)^*$

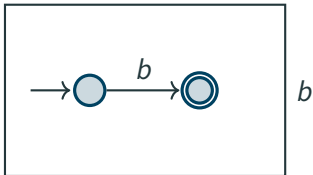
AUTOMAT FÜR $((a + b)b)^*$



AUTOMAT FÜR $((a + b)b)^*$

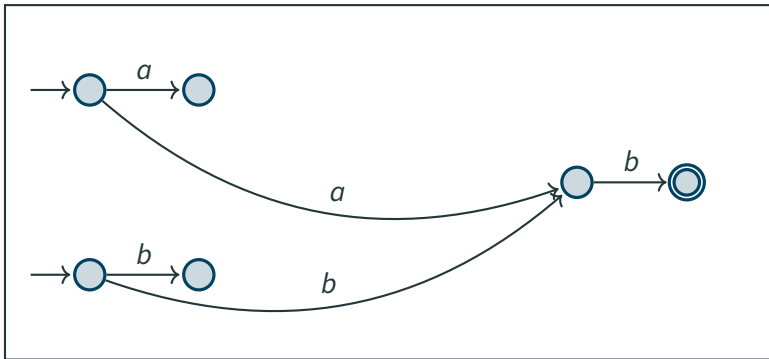


$a + b$



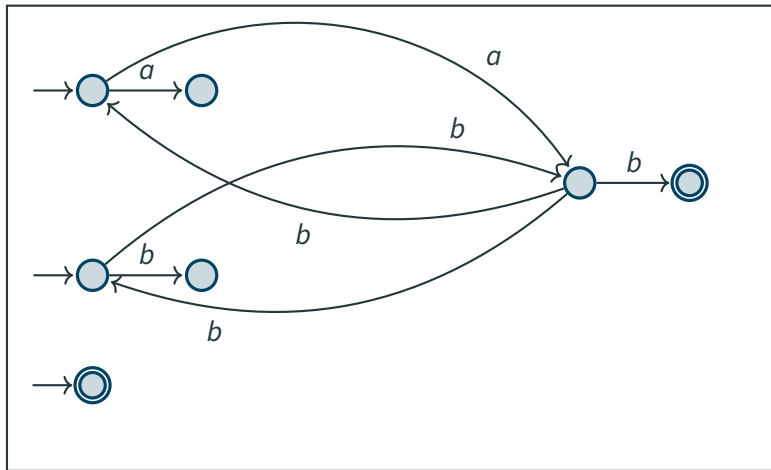
b

AUTOMAT FÜR $((a + b)b)^*$



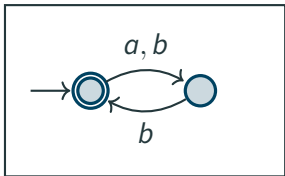
$(a + b)b$

AUTOMAT FÜR $((a + b)b)^*$



$((a + b)b)^*$

AUTOMAT FÜR $((a + b)b)^*$



$((a + b)b)^*$ (vereinfacht)

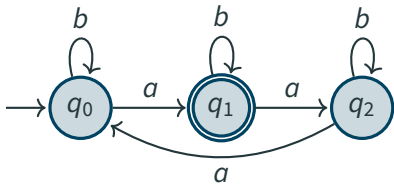
Satz (Kleene - Rückrichtung)

Für jeden nichtdeterministischen endlichen Automaten A gibt es einen regulären Ausdruck E mit

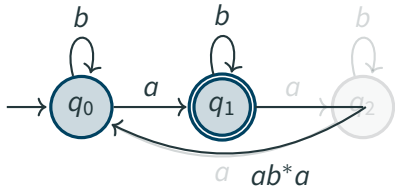
$$L(A) = L(E).$$

Elimination von Zuständen

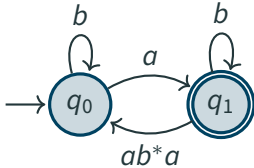
NEA $\rightsquigarrow$ RE (STARTZUSTAND IST NICHT AKZEPTIEREND)



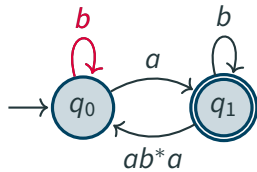
NEA $\rightsquigarrow$ RE (STARTZUSTAND IST NICHT AKZEPTIEREND)



NEA $\rightsquigarrow$ RE (STARTZUSTAND IST NICHT AKZEPTIEREND)

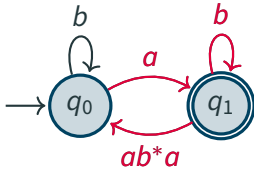


NEA $\rightsquigarrow$ RE (STARTZUSTAND IST NICHT AKZEPTIEREND)



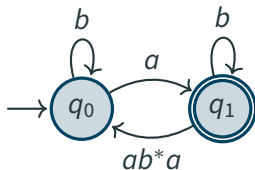
b

NEA $\rightsquigarrow$ RE (STARTZUSTAND IST NICHT AKZEPTIEREND)



$$b + (ab^*ab^*a)$$

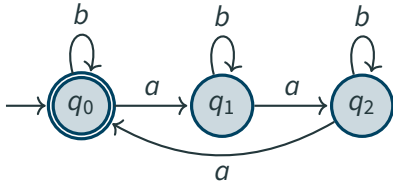
NEA $\rightsquigarrow$ RE (STARTZUSTAND IST NICHT AKZEPTIEREND)



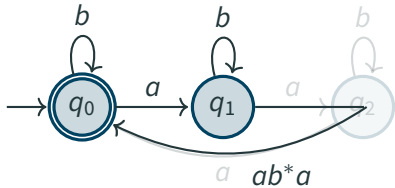
$$(b + (ab^*ab^*a))^*ab^*$$

- Annahme: Ein Startzustand und ein anderer akzeptierender Zustand
- Algorithmus: Beschrifte Transitionen mit regulären Ausdrücken und eliminiere Zustände bis nur Startzustand und akzeptierender Zustand übrig sind.
- Lies Ergebnis ab.

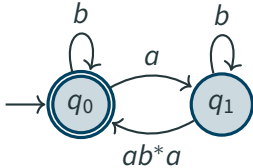
NEA $\rightsquigarrow$ RE (STARTZUSTAND IST AKZEPTIEREND)



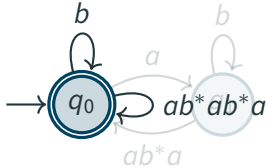
NEA $\rightsquigarrow$ RE (STARTZUSTAND IST AKZEPTIEREND)



NEA $\rightsquigarrow$ RE (STARTZUSTAND IST AKZEPTIEREND)

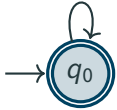


NEA $\rightsquigarrow$ RE (STARTZUSTAND IST AKZEPTIEREND)



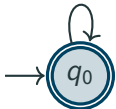
NEA $\rightsquigarrow$ RE (STARTZUSTAND IST AKZEPTIEREND)

$b + ab^*ab^*a$



NEA $\rightsquigarrow$ RE (STARTZUSTAND IST AKZEPTIEREND)

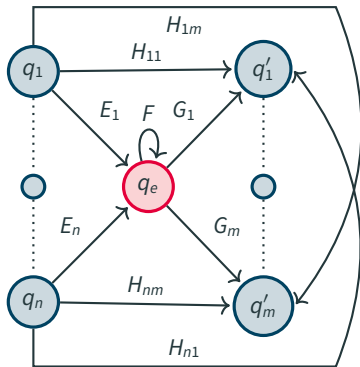
$b + ab^*ab^*a$



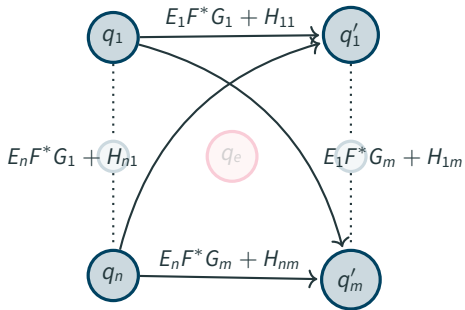
$(b + ab^*ab^*a)^*$

- Annahme: Ein Startzustand, der auch gleichzeitig einziger akzeptierender Zustand ist.
- Algorithmus: Beschrifte Transitionen mit regulären Ausdrücken und eliminiere Zustände bis nur der Start-/akzeptierender Zustand übrig ist.
- Lies Ergebnis ab.

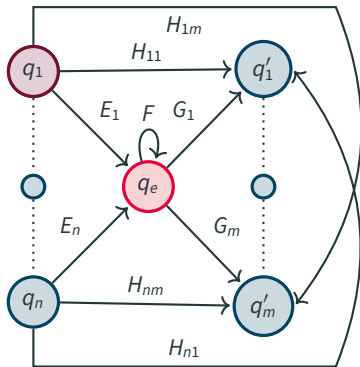
ELIMINIERE q_e (NICHT START UND NICHT AKZEPTIEREND)



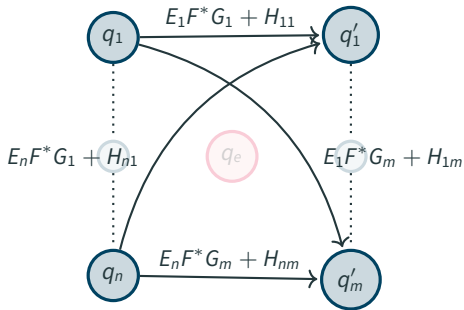
Elimination von q_e



ELIMINIERE q_e (NICHT START UND NICHT AKZEPTIEREND)

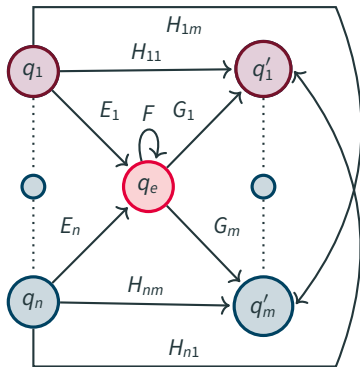


Elimination von q_e

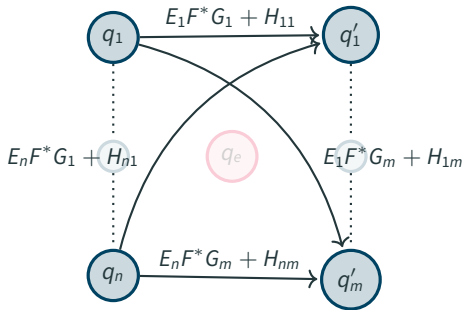


- Für einen Vorgänger q_1

ELIMINIERE q_e (NICHT START UND NICHT AKZEPTIEREND)

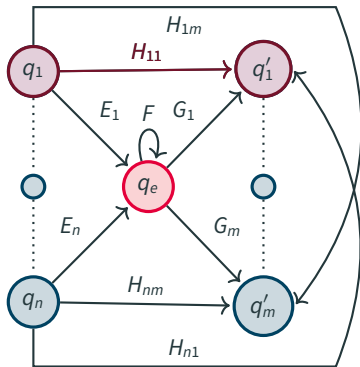


Elimination von q_e

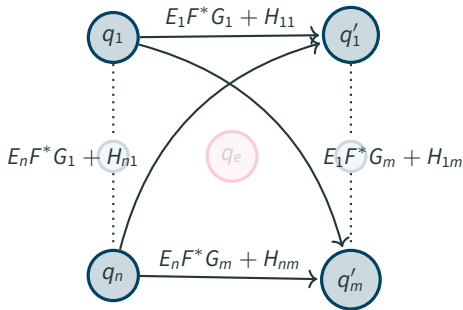


- Für einen Vorgänger q_1 und Nachfolger q'_1 kann es

ELIMINIERE q_e (NICHT START UND NICHT AKZEPTIEREND)

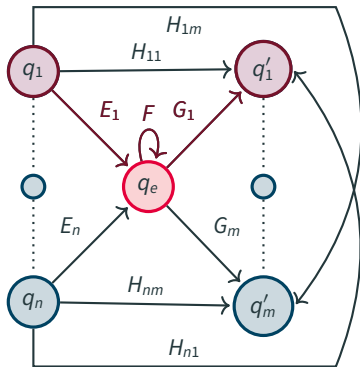


Elimination von q_e

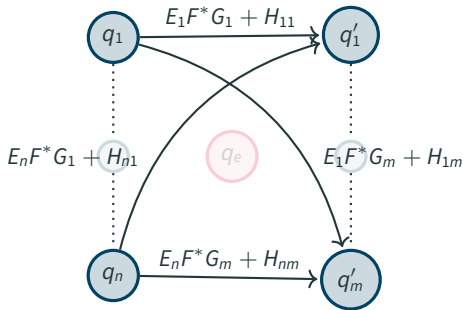


- Für einen Vorgänger q_1 und Nachfolger q'_1 kann es
 - eine direkte Transition mit Beschriftung H_{11} geben oder

ELIMINIERE q_e (NICHT START UND NICHT AKZEPTIEREND)

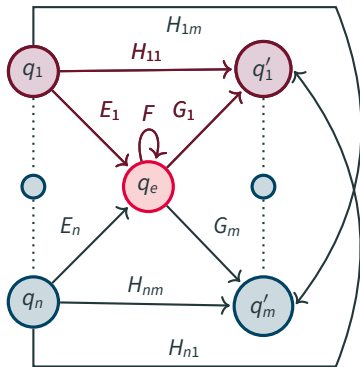


Elimination von q_e

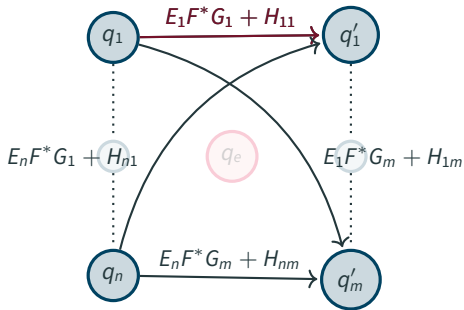


- Für einen Vorgänger q_1 und Nachfolger q'_1 kann es
 - eine direkte Transition mit Beschriftung H_{11} geben oder
 - einen Weg über q_e , der den Ausdruck $E_1 F^* G_1$ liefert.

ELIMINIERE q_e (NICHT START UND NICHT AKZEPTIEREND)

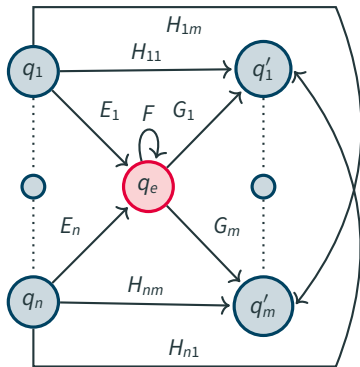


Elimination von q_e

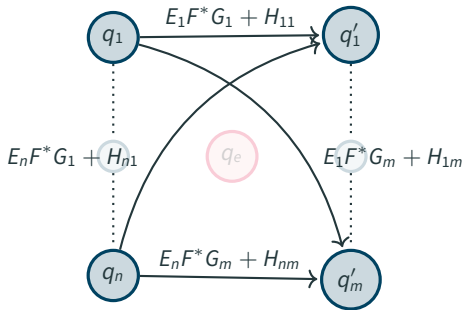


- Nach Elimination von q_e werden beide Wege durch eine mit $E_1F^*G_1 + H_{11}$ beschriftete Transition ersetzt.

ELIMINIERE q_e (NICHT START UND NICHT AKZEPTIEREND)

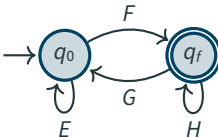
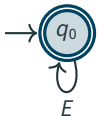


Elimination von q_e



- Das wird für alle Paare von Vorgängern und Nachfolgern von q_e gemacht.

1. Beschrifte Transitionen mit regulären Ausdrücken. Dabei entsprechen fehlende Transitionen dem Ausdruck $\emptyset$.
2. Für jede Kombination von Startzustand q_0 und akzeptierendem Zustand q_f
 - eliminieren wir alle Zustände außer q_0 und q_f
 - und erhalten zwei mögliche Ergebnisse:

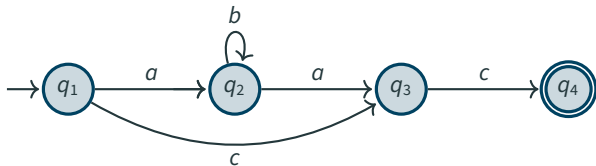
Bedingung	$q_0 \neq q_f$	$q_0 = q_f$
Automat		
Regulärer Ausdruck	$(E + FH^*G)^*FH^*$	E^*

Der reguläre Ausdruck für die NEA ist die Vereinigung aller Ausdrücke aus Schritt zwei

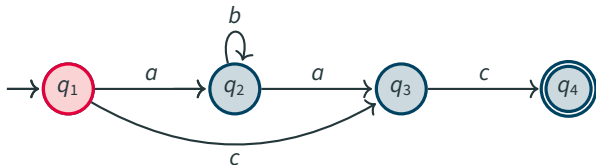
Lösen von Gleichungen



- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



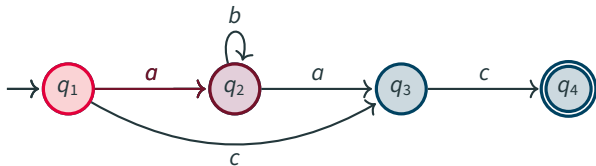
- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



Beispiel

X_1

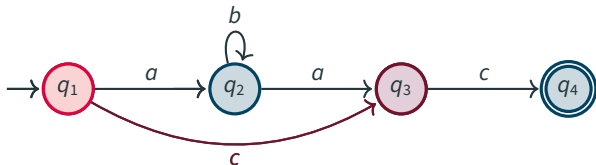
- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



Beispiel

$$X_1 = aX_2$$

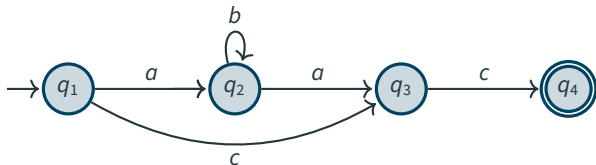
- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



Beispiel

$$X_1 = aX_2 + cX_3$$

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems

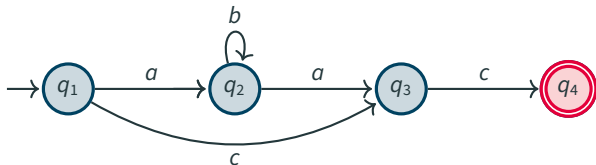


Beispiel

$$X_1 = aX_2 + cX_3 \quad X_2 = bX_2 + aX_3$$

$$X_3 = cX_4 \quad X_4 =$$

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems

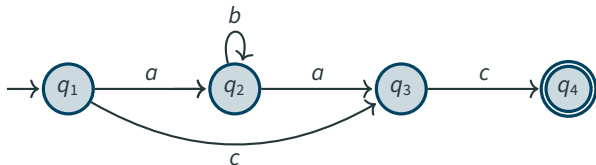


Beispiel

$$X_1 = aX_2 + cX_3 \quad X_2 = bX_2 + aX_3$$

$$X_3 = cX_4 \quad X_4 = \varepsilon \text{ (} q_4 \text{ ist akzeptiert)}$$

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



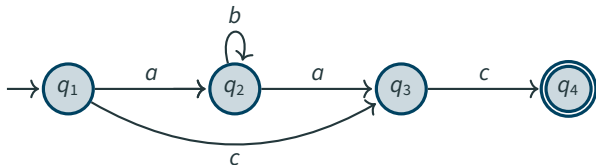
Beispiel

$$X_1 = aX_2 + cX_3 \quad X_2 = bX_2 + aX_3$$

$$X_3 = cX_4 \quad X_4 = \varepsilon$$

Setze $X_4 = \varepsilon$ in Gleichung für X_3 ein.

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



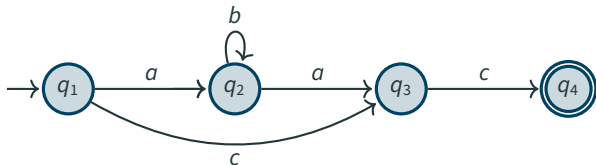
Example

$$X_1 = aX_2 + cX_3 \quad X_2 = bX_2 + aX_3$$

$$X_3 = c\varepsilon = c \quad X_4 = \varepsilon$$

Setze $X_3 = c$ in Gleichung für X_2 ein.

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



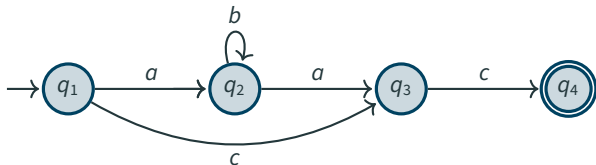
Example

$$X_1 = aX_2 + cX_3 \quad X_2 = bX_2 + ac$$

$$X_3 = c \quad X_4 = \varepsilon$$

rekursive Gleichung

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



Example

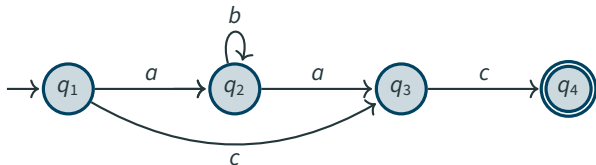
$$X_1 = aX_2 + cX_3 \quad X_2 = bX_2 + ac$$

$$X_3 = c \quad X_4 = \varepsilon$$

Lemma (Ardens Lemma)

Seien U, V, X Sprachen mit $\varepsilon \notin U$. Dann hat die Gleichung $X = UX \cup V$ die Lösung $X = U^*V$.

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



Example

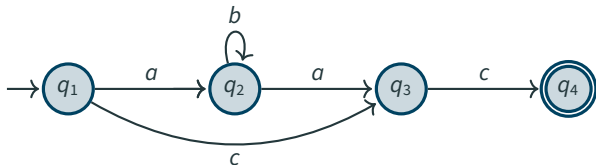
$$X_1 = aX_2 + cX_3 \quad X_2 = bX_2 + ac$$

$$X_3 = c \quad X_4 = \varepsilon$$

Folgerung (Ardens Lemma II)

Seien E, F, X reguläre Ausdrücke mit $\varepsilon \notin L(E)$. Dann hat die Gleichung $X = EX + V$ die Lösung $X = E^*F$.

- Für jedem Zustand q_i führen wir eine Variabel X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



Example

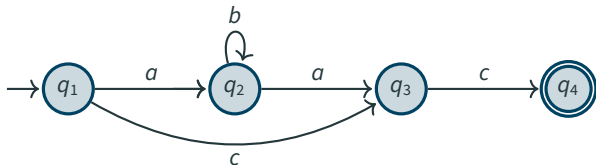
$$X_1 = aX_2 + cX_3 \quad X_2 = b^*ac$$

$$X_3 = c \quad X_4 = \varepsilon$$

Folgerung (Ardens Lemma II)

Seien E, F, X reguläre Ausdrücke mit $\varepsilon \notin L(E)$. Dann hat die Gleichung $X = EX + V$ die Lösung $X = E^*F$.

- Für jedem Zustand q_i führen wir eine Variable X_i ein für den regulären Ausdruck, der alle Wörter beschreibt, die von q_i in einen akzeptierenden Zustand führen.
- Ableiten und Lösen eines Gleichungssystems



Example

$$X_1 = ab^*ac + cc$$

$$X_2 = b^*ac$$

$$X_3 = c$$

$$X_4 = \varepsilon$$

Regulärer Ausdruck für Automat steht in Variable für Startzustand oder deren Vereinigung falls es mehr als einen Startzustand gibt.

Sei $A = (Q, \Sigma, \delta, Q_0, F)$ ein NEA.

1. Für jeden Zustand $q_i \in Q$ führe Variable X_i und eine Gleichung:
 - 1.1 Seien $q_i \xrightarrow{a_1} q_{i_1}, \dots, q_i \xrightarrow{a_m} q_{i_m}$ die Transitionen, die in q_i starten.
 - 1.2 Falls $q_i \notin F$ (q_i ist nicht akzeptierend), führe die Gleichung ein

$$X_i = a_1 X_{i_1} + \dots + a_m X_{i_m}.$$

- 1.3 Falls $q_i \in F$ (q_i ist akzeptierend), führe die Gleichung ein

$$X_i = a_1 X_{i_1} + \dots + a_m X_{i_m} + \varepsilon.$$

2. Das System wird durch Substitution und Ardens Lemma gelöst.
3. Der reguläre Ausdruck für $L(A)$ ist die Vereinigung (+) aller Lösungen für alle Startzustände.